CSE 211 — Theory of Computation

Time and Space Complexity: P, NP, NP-Completeness, PSPACE

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Atif Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — NP-completeness proofs and class relationships
Lesson 23 — The Classes P and NP, Certificates — time complexity, verifiers, examples
Lesson 24 — Reductions, NP-Completeness, SAT and Cook's Theorem — polynomial reductions, NP-hard, the seven rule families
Lesson 25 — NP-Completeness Proofs — 3-colourability, clique, subgraph isomorphism, partition, Hamiltonian cycle
Lesson 26 — Space Complexity: PSPACE, Savitch, the Big Picture — SPACE/NSPACE, Savitch's theorem, relationships, Venn diagrams
Quick Notes — Complexity — revision bullets + key diagrams

NOTE Slide numbers refer to the PDF pages of Lecture - 8 (complexity).pdf (51 pages) and Lecture - 9 (space_complexity).pdf (20 pages). The reductions of Lesson 25 (3-SAT → 3-colouring, independent set → clique → subgraph isomorphism, Subset Sum → Partition) were verified by a program on random instances (satisfiable ⇔ 3-colourable etc.). The 3-colouring reduction is the standard one from the textbook (not in the slides) and is marked as such. Practice provenance: Actual = taken from a TF paper; Adapted = TF question with changed details; Constructed = new question in TF style.

Exam Map — What the TF Papers Ask

Session	Complexity questions
2023-24	Q12(i) Venn diagram: P, NP, NP-complete, recursive, RE (8) · Q12(ii) Relationship between PSPACE and NPSPACE; justify (7) · Q13 Cook's theorem; build the Boolean clause for δ(q,t) = {(q1,t1,R),(q2,t2,L)} with head at cell i, step j (15)
2022-23	Q13(i) How Cook's theorem shows all of NP reduces to SAT in polynomial time (8) · Q13(ii) Relationship between PSPACE and NPSPACE; justify (7)
2021-22	Q12 How Cook's theorem reduces all of NP to SAT; why poly algorithm for an NPC problem ⇒ P = NP (15) · Q13 Believed relationship: P vs NP, P vs PSPACE, PSPACE vs NPSPACE (15)
2020 (Jan term)	Q1(a) Certificate showing Dominating Set is in NP (6) · Q12(a) State Cook's theorem; why a poly algorithm for an NP-complete problem gives P = NP (7) · Q12(b) Venn diagram of P, NP, PSPACE, NPSPACE, EXPTIME with justification (8)
2019-20	Q13 T/F: SAT in P ⇒ P = NP; NP ⊇ PSPACE; PSPACE = NPSPACE (15)
2017-18	Q1(a) Define the Boolean satisfiability problem with examples (6) · Q1(b) Given 3-SAT is NP(-complete), prove graph 3-colourability is NP-complete (20) · Q1(c) 3-colour graph of the reduction for (x1 ∨ ¬x2 ∨ ¬x3) ∧ (x1 ∨ x3 ∨ x4) (9) · Q12 Prove subgraph isomorphism is NP-complete (15) · Q13(a) Define P, NP, PSPACE, NPSPACE (6) · Q13(b) Believed relationship: P vs NP, PSPACE vs NPSPACE, NP vs PSPACE (9)

EXAM FOCUS Two things recur in every paper since 2017-18: (1) a theory question on **Cook's theorem / P = NP** (2023-24 Q13 with the clause for a transition rule, 2022-23 Q13(i), 2021-22 Q12, Jan-2020 Q12(a)) and (2) a **class-relationship question** (2023-24 Q12, 2022-23 Q13(ii), 2021-22 Q13, 2019-20 Q13 true/false, Jan-2020 Q12(b) Venn diagram, 2017-18 Q13). 2017-18 also asked two full NP-completeness proofs (3-colourability, subgraph isomorphism) and 'Dominating Set is in NP' (Jan-2020). Learn Lesson 26's relationship table, the Cook proof outline, and the proof template of Lesson 25.

Lesson 23 — The Classes P and NP, Certificates

Lecture 8, slides 2–8

We study decidable problems and ask which run in time polynomial in the input length. Problems solvable in polynomial time on a computer are exactly those solvable in polynomial time on a TM; polynomial = tolerable, exponential = only small instances (sl. 2).

1. The Classes (sl. 3–7)

- A TM has **time complexity T(n)** if on every input of length n it halts after at most T(n) moves.
- **P** = languages decided by a deterministic TM in polynomial time (MST, shortest path, edit distance, max flow, sl. 4).
- **NP** = languages decided by a nondeterministic TM in which no branch has more than T(n) moves, T a polynomial — equivalently, problems whose solutions can be **checked in polynomial time** (SAT, Hamiltonian path, TSP, vertex cover, independent set, + everything in P, sl. 5–6).
- P ⊆ NP (a deterministic TM is an NTM with no choices). Whether P = NP is among the deepest open questions of mathematics: an NTM can guess exponentially many solutions and check each in polynomial time in parallel (sl. 7).

P and NP are defined for decision problems (yes/no). An optimisation problem becomes a decision problem by adding a bound k ('is there a solution of value ≥ k?'); if the decision version is in P, binary search over k finds the optimum in polynomial time (sl. 8).

2. NP via Certificates (textbook)

A language L is in NP iff there is a polynomial-time **verifier** V and a polynomial p such that $x \in L \Leftrightarrow \exists$ certificate c, $|c| \le p(|x|)$, with V(x, c) accepting. To show a problem is in NP: (1) say what the certificate is (polynomial size); (2) describe a polynomial-time check.

> **EXAM-READY ANSWER** Dominating Set is in NP (Jan-2020 Q1(a)). Instance: graph G = (V, E) and integer k; question: is there a set D ⊆ V with $|D| \le k$ such that every vertex is in D or adjacent to a vertex of D? **Certificate**: the set D itself (at most |V| vertex names, polynomial size). **Verifier**: (1) check $|D| \le k$; (2) for every vertex $v \in V$ check that $v \in D$ or some edge (v, d) with $d \in D$ exists — $O(|V| \cdot |D|) \le O(n^2)$ steps with an adjacency matrix. Accept iff every check succeeds. A yes-instance has such a D (the certificate), and a no-instance has no certificate that passes, so the language is decided by the NTM that guesses D and runs the verifier in polynomial time: Dominating Set ∈ NP.

Practice — Lesson 23

(a) Actual — Jan-2020 Q1(a). (b) Constructed.

(a) Given an instance of the Dominating Set problem, describe how the certificate can determine whether the Dominating Set problem is in NP. (6)

(b) Show that Hamiltonian Path and Vertex Cover are in NP (certificate and verifier).

✓ Solution

(a) EXAM-READY ANSWER above.

(b) Hamiltonian Path: certificate = an ordering v1…vn of all vertices; verifier checks it is a permutation (each vertex once) and that every (vi, vi+1) is an edge — $O(n^2)$. Vertex Cover (G, k): certificate = a set C of ≤ k vertices; verifier checks $|C| \le k$ and that every edge has an endpoint in C — O(|E|).

Lesson 24 — Reductions, NP-Completeness, SAT and Cook's Theorem

Lecture 8, slides 9–28

1. Polynomial-Time Reductions and NP-Completeness (sl. 9–16)

A polynomial-time reduction maps, in polynomial time, every instance of P1 to an instance of P2 with the same answer (sl. 9). L is **NP-complete** if (1) L ∈ NP and (2) every L′ ∈ NP reduces to L in polynomial time. L is **NP-hard** if (2) holds but L need not be in NP (sl. 15).

MEMORIZE If P1 is NP-complete, P2 ∈ NP and P1 reduces to P2 in polynomial time, then P2 is NP-complete (sl. 12): every L ∈ NP reduces to P1, compose with P1 → P2. NOTE THE DIRECTION: reduce the KNOWN hard problem TO the new problem (sl. 14).

If some NP-complete problem is in P then P = NP: every L ∈ NP reduces to it in polynomial time, so L ∈ P (sl. 13). Procedure to prove P2 NP-complete: (1) P2 ∈ NP (certificate); (2) pick an NP-complete P1 and give a polynomial reduction P1 → P2 with 'P1 yes ⇔ P2 yes' proved in both directions.

Hard problems (NP-complete)	Easy problems (in P)
3SAT	2SAT, HORN SAT
TRAVELING SALESMAN PROBLEM	MINIMUM SPANNING TREE
LONGEST PATH	SHORTEST PATH
3D MATCHING	BIPARTITE MATCHING
KNAPSACK	UNARY KNAPSACK
INDEPENDENT SET	INDEPENDENT SET on trees
INTEGER LINEAR PROGRAMMING	LINEAR PROGRAMMING
RUDRATA PATH	EULER PATH
BALANCED CUT	MINIMUM CUT

Figure 24.1 — Easy vs hard problems (sl. 11)

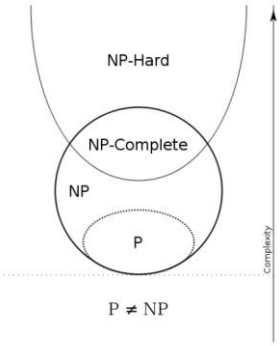


Figure 24.2 — Likely relation among P, NP, NP-complete, NP-hard (sl. 16)

2. SAT and Cook's Theorem (sl. 17–28)

A Boolean formula uses variables, ∧, ∨, ¬ and parentheses; a truth assignment satisfies it if it makes it true; SAT = {φ | φ is satisfiable}. Example (satisfiable): (x ∨ y) ∧ (¬x ∨ z); not satisfiable: x ∧ ¬x. 3-SAT: CNF with exactly three literals per clause.

EXAM-READY ANSWER Cook's theorem: SAT is NP-complete (sl. 20–28). SAT ∈ NP: guess an assignment and evaluate. Hardness: let L ∈ NP be decided by a single-tape NTM M in time p(n) on every branch. For an input w, |w| = n, build a formula F(M, w) in polynomial time that is satisfiable iff some branch of M accepts w. F(M, w) does not analyse the branches; it lets every truth assignment describe one possible computation as a (p(n)+1) × (p(n)+1) tableau of configurations. Variables: s(q, j) — M is in state q at step j; l_ij — the head is at cell i at step j; c(i, j, t) — cell i holds symbol t at step j; u(i, j) — cell i is unchanged from step j to j + 1. F(M, w) is the AND of seven families: (1) exactly one state at each step; (2) the head in exactly one cell at each step; (3) exactly one symbol in each cell at each step; (4) an unchanged cell keeps its symbol: c(i, j, t) ∧ u(i, j) → c(i, j+1, t); (5) the transition rule is obeyed (below); (6) the initial configuration — one formula: s(q0, 0) ∧ l_00 ∧ c(0,0,w1) ∧ … ∧ c(n−1,0,wn) ∧ c(n,0,B) ∧ … ; (7) acceptance: s(q_accept, p(n)). Size: rule 5 dominates, O(|Q||Γ|p(n)³) clauses — polynomial. Then w ∈ L ⇔ F(M, w) satisfiable, so L reduces to SAT in polynomial time. Hence every problem in NP reduces to SAT and SAT is NP-complete.

EXAM-READY ANSWER Rule 5 for δ(q, t) = {(q1, t1, R), (q2, t2, L)}, head at cell i at step j (2023-24 Q13; assume the head is not at the leftmost or rightmost cell, so 0 < i < p(n)): if M is in state q, the head is at i and cell i holds t at step j, then all other cells are unchanged and EITHER the machine moves right: new state q1, head at i + 1, cell i holds t1, OR it moves left: state q2, head at i − 1, cell i holds t2:

$$[\, s(q, j) \wedge l_{ij} \wedge c(i, j, t) \,] \rightarrow \bigwedge_{k \neq i} u(k, j) \;\wedge$$
$$[\, (\, s(q1, j+1) \wedge l_{i+1,\, j+1} \wedge c(i, j+1, t1)\,) \;\vee\; (\, s(q2, j+1) \wedge l_{i-1,\, j+1} \wedge c(i, j+1, t2)\,)\,]$$

Written in CNF (needed for the reduction to 3-SAT): replace A → B by ¬A ∨ B, i.e. the clause ¬s(q,j) ∨ ¬l_ij ∨ ¬c(i,j,t) ∨ (…), and distribute: each conjunct u(k, j) gives a clause ¬s(q,j) ∨ ¬l_ij ∨ ¬c(i,j,t) ∨ u(k,j); the disjunction of the two triples distributes into 3 × 3 = 9 clauses ¬s ∨ ¬l ∨ ¬c ∨ x ∨ y with x from the first triple and y from the second. Boundary cases (slide): φ(k) = i − 1 if d_k = L and i > 0; i if d_k = L and i = 0, or d_k = R and i = p(n); i + 1 if d_k = R and i < p(n).

EXAM-READY ANSWER Why a polynomial algorithm for an NP-complete problem implies P = NP (2021-22 Q12, Jan-2020 Q12(a)): let Π be NP-complete with a polynomial algorithm A. Take any L ∈ NP. By the definition of NP-completeness there is a polynomial-time reduction f from L to Π. Decide x ∈ L by computing f(x) (polynomial time) and running A on f(x) (polynomial in |f(x)|, which is polynomial in |x|): total polynomial. Hence L ∈ P for every L ∈ NP, so NP ⊆ P; with P ⊆ NP this gives P = NP.

Exam Approach — 'How Cook's Theorem Shows All of NP Reduces to SAT'

1. State SAT ∈ NP in one sentence.
2. Fix L ∈ NP, NTM M with p(n) steps; the tableau idea.
3. List the variables and the seven formula families with one line each; mention the polynomial size.
4. Conclude: w ∈ L ⇔ F(M, w) satisfiable; hence L ≤p SAT, so SAT is NP-complete.

Practice — Lesson 24

(a) Actual — 2023-24 Q13. (b) Actual — 2022-23 Q13(i), 2021-22 Q12. (c) Actual — Jan-2020 Q12(a). (d) Actual — 2017-18 Q1(a).

(a) Briefly explain how Cook's theorem shows that all problems in NP are reducible to the satisfiability problem. Suppose an NTM has a transition rule δ(q, t) = {(q1, t1, R), (q2, t2, L)}. Construct the corresponding rule in the Boolean satisfiability formula according to Cook's construction if the tape head is at the i-th cell in the j-th step (not at the leftmost or rightmost non-blank cells). (8 + 7)

(b) How does Cook's theorem show that all problems in NP are polynomial-time reducible to SAT? (8)

(c) State Cook's theorem. Explain why finding a polynomial-time algorithm for an NP-complete problem implies P = NP. (7)

(d) Define the Boolean satisfiability problem with necessary examples. (6)

✓ Solution

(a)–(c) EXAM-READY ANSWERS above (the tableau proof; the rule-5 formula; the P = NP argument).

(d) SAT: given a Boolean formula φ over variables x1, …, xn with ∧, ∨, ¬, decide whether some truth assignment makes φ true. (x1 ∨ x2) ∧ (¬x1 ∨ x3) is satisfiable (x1 = T, x3 = T); (x1) ∧ (¬x1) is not. It is in NP: the assignment is the certificate.

Lesson 25 — NP-Completeness Proofs

Lecture 8, slides 29–51 + textbook (3-colourability)

1. 3-Colourability Is NP-Complete (★★★ 2017-18 Q1(b)–(c); textbook construction)

3-COLOURABILITY = {G | the vertices of G can be coloured with 3 colours so that adjacent vertices differ}. **In NP**: certificate = the colouring; check every edge — O(|E|). **Hardness**: reduce 3-SAT (NP-complete) to it. Given φ with variables x1…xn and clauses C1…Cm build G:

1. **Palette triangle** T, F, B (true, false, base): three mutually adjacent vertices — they get three different colours; call them true, false, base.
2. **Variable gadget**: for each xi vertices xi and ¬xi, adjacent to each other and both adjacent to B. So in any 3-colouring one of xi, ¬xi gets the colour of T and the other the colour of F (neither can have B's colour).
3. **Clause gadget** for Cj = (a ∨ b ∨ c): an OR-gadget on two inputs a, b — a triangle {g1, g2, o1} with g1 adjacent to a and g2 adjacent to b; o1 is the output. A second OR-gadget (triangle {h1, h2, o2}) takes inputs o1 and c (h1 adjacent to o1, h2 adjacent to c); the final output o2 is joined to both F and B, so it must get T's colour. An OR-gadget's output can be coloured T's colour iff at least one input is coloured T's colour.
4. The graph has 3 + 2n + 6m vertices — polynomial.

Correctness: (⇒) given a satisfying assignment colour xi with T's colour iff xi is true (¬xi the opposite); in every clause at least one literal is coloured true, so the gadget's outputs can be coloured with 'true' (adjacent to F and B, so g2 must be coloured T's colour) — the colouring extends. (⇐) given a 3-colouring read xi = true iff xi has T's colour; each clause output g2 is adjacent to F and B, so it is coloured T's colour, which is only possible if some input literal has that colour — the clause is satisfied.

> **EXAM-READY ANSWER** (c) The graph for (x1 ∨ ¬x2 ∨ ¬x3) ∧ (x1 ∨ x3 ∨ x4) — produced by the construction (checked by program): 23 vertices, 39 edges. Palette: T, F, B (triangle). Variables x1…x4 and ¬x1…¬x4, each pair adjacent and joined to B. Clause 1: gadget with inputs x1, ¬x2, ¬x3 (gate vertices c1a, c1b, c1o1, c1c, c1d, c1o2), c1o2 joined to F and B. Clause 2: inputs x1, x3, x4 (c2a … c2o2), c2o2 joined to F and B. The formula is satisfiable (e.g. x1 = true) and the graph is 3-colourable — the program confirms the equivalence on random formulas (0 mismatches in 60 trials) and that a small unsatisfiable formula gives a non-3-colourable graph.

2. Clique and Subgraph Isomorphism (★★★ 2017-18 Q12; sl. 41–46)

Clique (G, k): is there a set of ≥ k mutually adjacent vertices? In NP (certificate = the vertex set; check all pairs). Reduction from INDEPENDENT SET (known NP-complete): given (G, k) let G′ be the **complement graph** (same vertices, edge iff NOT an edge of G) and k′ = k. G′ has a clique of size k ⇔ G has an independent set of size k (sl. 43).

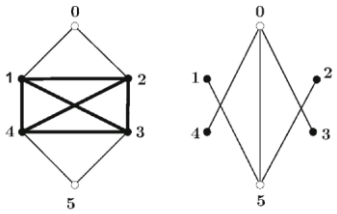


Figure 25.1 — Independent set ↔ clique in the complement graph (sl. 44)

EXAM-READY ANSWER Subgraph Isomorphism is NP-complete (2017-18 Q12; same idea as Maximum Common Subgraph, sl. 45–46). Problem: given graphs G1, G2, does G1 contain a subgraph isomorphic to G2 (a subset of nodes whose induced edges, with a one-to-one correspondence of nodes, form an exact copy of G2)? **In NP**: certificate = the subset of nodes of G1 and the bijection to the nodes of G2; check that every edge of G2 maps to an edge of G1 and every non-edge to a non-edge — O(|V2|²) (for 'exact copy' check the induced subgraph). **NP-hard**: reduce CLIQUE (NP-complete): given (G, k) output G1 = G and G2 = the complete graph K_k. Computable in polynomial time (k ≤ |V|). Correctness: G has a clique of size k ⇔ G contains K_k as a subgraph ⇔ G1 contains a subgraph isomorphic to G2. (⇒) the clique's k vertices induce K_k; (⇐) a copy of K_k consists of k mutually adjacent vertices of G. Hence Subgraph Isomorphism is NP-complete. (Verified by program on random graphs.)

EXAM-READY ANSWER Maximum Common Subgraph (sl. 45–46): reduction from Clique with G1 = G, G2 = complete graph on |V| vertices, k′ = k: any common subgraph of size ≥ k must be complete (G2 is), i.e. a k-clique of G; conversely a k-clique is a common subgraph of size k.

3. Partition from Subset Sum (sl. 47–51)

Subset Sum: nonnegative a1…am and t — is there S with $\Sigma_{i\in S}$ ai = t? Partition: is there S with $\Sigma_{i\in S}$ ai = $\Sigma_{j\notin S}$ aj? Partition ∈ NP (guess S, add). Reduction Subset Sum → Partition: let a = Σ ai. Case 2t ≥ a: add a_{m+1} = 2t − a. If S sums to t then T′ = {1..m+1} − S sums to (a − t) + (2t − a) = t — a partition. Conversely if S creates a partition then each half sums to (a + (2t − a))/2 = t, and the half not containing a_{m+1} (or the other half minus a_{m+1}) is a solution of Subset Sum. Case 2t ≤ a: add a − 2t (sl. 50–51).

4. Directed Hamiltonian Cycle (sl. 30–40)

DIRHAMCYCLE ∈ NP (certificate = the cycle). Hardness: reduce 3-SAT. For xi build a row of 3m + 3 vertices with edges both ways — a Hamiltonian cycle must traverse the row straight through: left→right = xi true, right→left = false. Add s, t and wiring s → both ends of row 1 → … → both ends of row n → t → s. For each clause Cj add a vertex cj and, using positions 3j and 3j+1 in every row, a detour vi,3j → cj → vi,3j+1 (available when the row runs left→right for a positive literal, right→left for a negative one). A cycle covering all cj exists iff some literal of every clause is true (sl. 32–40).

Exam Approach — NP-Completeness Proof

1. Problem ∈ NP: certificate + polynomial verifier.
2. Choose a known NP-complete problem and say why it is natural (3-SAT for gadgets, Clique/IS for graph problems, Subset Sum for numbers).
3. Describe the construction (gadgets / complement / padding), polynomial size.
4. Prove both directions: instance of the known problem is yes ⇔ constructed instance is yes.

5. Conclude NP-complete.

TRAP Direction of the reduction! Reduce from the KNOWN NP-complete problem to the new one. Do not forget to show membership in NP — without it you only prove NP-hardness.

Practice — Lesson 25

(a) Actual — 2017-18 Q1(b)–(c). (b) Actual — 2017-18 Q12. (c) Constructed (sl. 43). (d) Constructed (sl. 47–51).

(a) With the knowledge that 3-SAT is NP, prove that graph 3-colourability is NP-complete. Then show the final 3-colour graph for (x1 ∨ ¬x2 ∨ ¬x3) ∧ (x1 ∨ x3 ∨ x4). (20 + 9)

(b) The subgraph isomorphism problem is: given graphs G1, G2, does G1 contain a copy of G2 as a subgraph? Prove that it is NP-complete. (15)

(c) Prove that Clique is NP-complete assuming Independent Set is.

(d) Prove that Partition is NP-complete assuming Subset Sum is.

✓ Solution

(a) §1 (construction, both directions, the graph for the formula: 23 vertices).

(b) EXAM-READY ANSWER in §2.

(c)–(d) §2 (complement graph) and §3 (padding element 2t − a).

Lesson 26 — Space Complexity: PSPACE, Savitch, the Big Picture

Lecture 9, slides 2–20

1. Definitions (sl. 2–6)

Space complexity of a deterministic TM that halts on every input: f(n) = the maximum number of tape cells scanned on any input of length n (for an NTM: the maximum over all branches). SPACE(f(n)) = languages decided by an O(f(n))-space DTM; NSPACE(f(n)) likewise for NTMs. **PSPACE** = ∪_k SPACE(n^k); **NPSPACE** = ∪_k NSPACE(n^k). Example: SAT can be solved in linear space (try all assignments one after another, reusing the space). Space seems more powerful than time because space can be reused, time cannot (sl. 6).

2. Relationships (sl. 7–10)

P ⊆ PSPACE and NP ⊆ NPSPACE: a TM that makes only polynomially many moves visits only polynomially many cells. If M uses polynomial space p(n) and accepts, it does so within $c^{1+p(n)}$ moves (c = s + t, s states, t tape symbols): otherwise an ID repeats (there are at most $s \cdot p(n) \cdot t^{p(n)}$ IDs) and the computation would loop (sl. 8–10). Together with Savitch: **P ⊆ NP ⊆ PSPACE = NPSPACE ⊆ EXPTIME**.

3. Savitch's Theorem (sl. 11–18)

EXAM-READY ANSWER Theorem: for f(n) ≥ n, NSPACE(f(n)) ⊆ SPACE(f²(n)); in particular PSPACE = NPSPACE (2022-23 Q13(ii), 2023-24 Q12(ii)). PSPACE ⊆ NPSPACE is trivial (a DTM is an NTM); we show NPSPACE ⊆ PSPACE. Trying NTM branches one by one would need to record all choices of a branch of length $c^{O(f(n))}$ — exponential space. Instead solve the reachability problem REACH(I, J, m): can the NTM go from ID I to ID J in at most m steps using f(n) space? Procedure: if m = 1 check directly; otherwise for every ID K of length f(n) test REACH(I, K, m/2) and then REACH(K, J, m/2), and accept if some K works. The two recursive calls are made one after the other, so only one chain of calls is active at a time: the recursion depth is $\log_2 m$, and each stack frame stores two IDs and a number, O(f(n)) cells. For an accepting computation of the NTM $m = c^{1+f(n)}$, so the depth is O(f(n)); total space O(f(n)) · O(f(n)) = O(f²(n)). The DTM calls REACH(I0, J, $c^{1+f(n)}$) for each accepting ID J using at most f(n) cells (enumerated systematically). For polynomial f the bound f² is polynomial, so NPSPACE = PSPACE.

4. The Big Picture (sl. 19–20)

Pair	Widely believed / known relationship	Justification
P and NP	P ⊆ NP; believed P ≠ NP (open)	every DTM is an NTM; NTM guesses exponentially many solutions; no one has a polynomial algorithm for any NP-complete problem
PSPACE and NPSPACE	equal (proved)	Savitch's theorem: NSPACE(f) ⊆ SPACE(f²)
NP and PSPACE	NP ⊆ PSPACE; believed NP ≠ PSPACE (open)	a polynomial-space machine can try all certificates of polynomial length one after the other, reusing space; equality unknown
P and PSPACE	P ⊆ PSPACE; believed P ≠ PSPACE (open)	polynomial time ⇒ polynomial space
PSPACE and EXPTIME	PSPACE ⊆ EXPTIME	a poly-space machine has ≤ $c^{1+p(n)}$ IDs, so it halts within exponential time

Pair	Widely believed / known relationship	Justification
EXPTIME and P	P ⊊ EXPTIME (time hierarchy theorem)	textbook

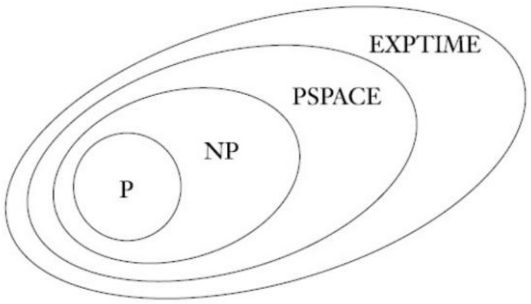


Figure 26.1 — Conjectured relation among classes of languages (sl. 19–20)

EXAM-READY ANSWER Venn diagram (Jan-2020 Q12(b), 2023-24 Q12(i)): nested regions, outermost first: all languages ⊃ RE ⊃ recursive (decidable) ⊃ EXPTIME ⊃ PSPACE = NPSPACE ⊃ NP ⊃ P; inside NP the NP-complete problems touch the boundary of NP away from P (if P ≠ NP, NP-complete ∩ P = ∅). Justify: P ⊆ NP ⊆ PSPACE = NPSPACE ⊆ EXPTIME ⊆ recursive ⊆ RE; the strictness of P ⊊ NP ⊊ PSPACE is believed, not proved; only P ⊊ EXPTIME is proved.

Exam Approach

1. Define each class in one line (P, NP, PSPACE, NPSPACE) — polynomial time/space, deterministic or nondeterministic.
2. For 'relationship' questions give: proven / believed, plus the one-line reason from the table.
3. True/false items: check the direction of every ⊆ and whether it is proved or only believed.

Practice — Lesson 26

(a) Actual — 2017-18 Q13(a)–(b). (b) Actual — 2021-22 Q13. (c) Actual — 2019-20 Q13. (d) Actual — Jan-2020 Q12(b), 2023-24 Q12. (e) Actual — 2022-23 Q13(ii), 2023-24 Q12(ii).

(a) Define the classes P, NP, PSPACE, NPSPACE. For P and NP; PSPACE and NPSPACE; NP and PSPACE: what is the widely believed relationship? Briefly justify. (6 + 9)

(b) What is the widely believed relationship between P and NP; P and PSPACE; PSPACE and NPSPACE? (15)

(c) True or false, with a brief reason: (i) if SAT is in P then P = NP; (ii) NP ⊇ PSPACE; (iii) PSPACE = NPSPACE. (15)

(d) Draw the Venn diagram of the widely believed relationships among P, NP, NP-complete, PSPACE, NPSPACE, EXPTIME (Jan-2020) / P, NP, NP-complete, recursive and RE languages (2023-24). Justify. (8)

(e) What is the relationship between PSPACE and NPSPACE? Justify briefly. (7)

✓ Solution

(a) Definitions: P = decided by a DTM in polynomial time; NP = by an NTM in polynomial time (equivalently verifiable in polynomial time); PSPACE / NPSPACE = decided by a DTM / NTM in polynomial space. Relationships: table in §4.

(b) P ⊆ NP, believed proper; P ⊆ PSPACE, believed proper; PSPACE = NPSPACE (Savitch).

(c) (i) TRUE — SAT is NP-complete, so a polynomial algorithm for it decides every $L \in NP$ in polynomial time (Lesson 24). (ii) FALSE (as far as is known) — the known inclusion is $NP \subseteq PSPACE$; $PSPACE \subseteq NP$ would put PSPACE-complete problems in NP and is believed false. (iii) TRUE — Savitch's theorem.

(d) EXAM-READY ANSWER in §4 (draw the nested ellipses; mark NP-complete inside NP, disjoint from P if $P \neq NP$).

(e) Equal: PSPACE = NPSPACE by Savitch's theorem ($NSPACE(f) \subseteq SPACE(f^2)$ and the square of a polynomial is a polynomial); the $\subseteq$ in one direction is trivial. Unlike time, determinism costs only a quadratic space blow-up.

Quick Notes — Complexity

Revision points for Lessons 23–26.

L23 · P and NP

- P = poly time DTM; NP = poly time NTM = polynomial verifier with certificate. P ⊆ NP; P = NP open.
- Certificate proofs: Dominating Set (the set D; check |D| ≤ k and every vertex dominated).

L24 · NP-complete, SAT

- NPC = in NP + every NP problem reduces. Reduce KNOWN-hard → new. NPC problem in P ⇒ P = NP.
- Cook: SAT ∈ NP; tableau $(p(n)+1)^2$; variables s, l, c, u; seven families; rule 5 $O(|Q||\Gamma|p(n)^3)$.
- Rule for δ(q,t) = {(q1,t1,R),(q2,t2,L)}: $[s(q,j) \wedge l_ij \wedge c(i,j,t)] \rightarrow \wedge_{k \neq i} u(k,j) \wedge [(s(q1,j+1) \wedge l_{i+1,j+1} \wedge c(i,j+1,t1)) \vee (s(q2,j+1) \wedge l_{i-1,j+1} \wedge c(i,j+1,t2))]$.

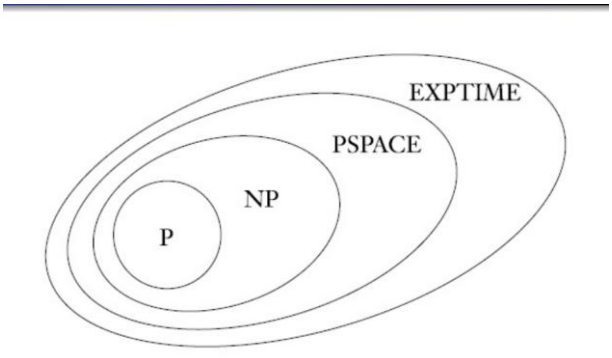
L25 · Reductions

- 3-colour from 3-SAT: palette T,F,B; variable pair adjacent to each other and B; OR-gadget per clause output joined to F and B; 3 + 2n + 6m vertices.
- Clique from IS: complement graph, k′ = k. Subgraph isomorphism from clique: G2 = K_k. Partition from Subset Sum: add |2t − a|.
- Always: in NP + correct direction + both directions of equivalence.

L26 · Space

- PSPACE = NPSPACE (Savitch: NSPACE(f) ⊆ SPACE(f^2); REACH(I,J,m) halving, depth log m, frame O(f)).
- P ⊆ NP ⊆ PSPACE = NPSPACE ⊆ EXPTIME; only P ⊊ EXPTIME proved; the others believed proper.
- T/F: SAT ∈ P ⇒ P = NP true; NP ⊇ PSPACE false (believed); PSPACE = NPSPACE true.

Key Diagrams to Reproduce



Conjectured relation among classes (Lecture 9, sl. 20)